ASP.NET MVC Dependency Injection Guide

1- Problem: Tight Coupling when Controller creates Repository using new. 2- Dependency: Any object required by another object to perform work. 3- Before DI: Controller -> Repository -> DbContext Each class creates the next one. 4- DI Idea: Receive dependencies instead of creating them. 5- Injection: Passing dependency from outside the class. 6- DI Container: Creates objects, resolves dependencies, manages lifetime. 7- Why Interface? Depend on IDepartmentRepository instead of DepartmentRepository. Allows replacing implementations without changing Controller. 8- Registration: builder.Services.AddScoped(); Meaning: When IDepartmentRepository is requested, provide DepartmentRepository. 9- Constructor Injection: public DepartmentController(IDepartmentRepository repo) 10- DbContext Injection: Preferred: public DepartmentRepository(MVCAppG04DbContext dbContext) Avoid: new MVCAppG04DbContext() 11- Bug: _dbContext = dbContext; Not: dbContext = dbContext; 12- Requests: DI does not change number of requests or responses. It only manages object creation. 13- Lifetimes: Transient = new object every resolution. Scoped = one object per request. Singleton = one object for entire application. 14- Why Scoped? DbContext is usually Scoped to isolate each request. 15- Testing: DI allows Mock repositories and easier unit testing. 16- Interview Definition: A class should not create its dependencies. Dependencies should be supplied from outside. 17- Benefits: Loose Coupling Easy Testing Maintainability SOLID Principles Cleaner Architecture